ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Encapsulation — Class Challenge Set

The challenges centre on a single small abstraction — a digital wallet that prevents double-spending — and ask you to enforce its invariants through encapsulation.

The domain: a wallet with no double-spending

We model a simplified wallet for a digital currency. Two abstractions are involved:

- `Money` : a unit of currency. Values of type `Money` carry a unique identifier and an amount. Once minted, a `Money` value represents a *capability* to spend that amount, but only once.
- `Wallet` : the authority that issues units of currency and tracks which ones have been spent. It maintains an internal record of spent identifiers (the actual record is never exposed to clients).

The crucial invariant is: **a given `Money` value can be spent at most once.** This must be guaranteed by the abstraction itself — clients should not be able to violate it, accidentally or otherwise.

The scenarios a client should be able to express are:

- start with a fresh, empty wallet;
- ask the wallet to issue (*mint*) new `Money` of a given amount;
- attempt to *spend* a `Money` , that is, mark it as consumed and obtain its amount in return, with the operation failing if the Money has already been spent
- *verify* whether a `Money` is still spendable, without spending it.

To keep the challenge tractable, you may assume that each Money value is atomic: it is spent in full or not at all, never in fractions.

What clients should *not* be able to do is fabricate a `Money` directly (bypassing the wallet's minting operation), mark a previously spent `Money` as unspent, or otherwise reach into the wallet's internal record of spent ids.

Throughout, the wallet is a purely functional value: every operation that changes its state returns a new wallet, and clients pass it explicitly through the chain of operations.

Part A — Haskell

Design the interface

Before writing any code, decide *what the module exposes*. Then translate the scenarios above into a Haskell module:

- *What types are exported?* Which of `Wallet` and `Money` are visible to clients? Are their data constructors exported, or only the type names? What is the difference?
- *Creating a wallet.* What is the signature of the operation that produces a fresh, empty wallet? Is it a function or a constant?
- *Issuing currency.* When the wallet issues new currency, what does the operation return? Just a `Money`? A new `Wallet` too? Why?
- *Spending.* When a client attempts to spend a `Money`, what should the operation return? Consider:
  - It can fail (the `Money` may already have been spent). How is failure communicated?
  - The wallet's state changes. How is the new state returned?
  - The client probably wants to know *how much* was spent. Is that part of the return value, or recoverable some other way?
- *Querying.* Should clients be able to ask "is this `Money` still spendable?" without spending it? Should they be able to ask "what is the amount of this `Money`?" — and is that operation safe to expose?
- *What is **not** in the interface?* What operations would *break* the invariant if exposed? What internal data must remain hidden?

Implement the interface

Now implement the `Wallet` module against the interface your group designed (or against the one we converged on in plenary, if you prefer). Choose internal representations for `Wallet` and `Money` that make the implementation straightforward.

A possible starting point, if you need one:

- `Money` carries a unique id and an amount.
- `Wallet` carries the set of spent ids and a counter for generating fresh ids.

Make sure the export list of your module exposes only the type names and the operations you decided on — *not* the data constructors. This is what separates the abstraction from a plain record.

Exercise the abstraction

Test it. Write a small client function that mints two `Money` values, spends one of them successfully, then attempts to spend the same `Money` a second time. Verify that the second attempt is rejected and that the first one returns the correct amount. The exact shape of this client code depends on the interface you designed — it should read naturally given your signatures. If it does not, the interface may be worth revisiting.

Try to break it. From a *client* module that imports your `Wallet` module, attempt the following:

- Construct a `Money` value directly with a fabricated id and amount (whatever your data constructor for `Money` is called). What does the compiler say?
- Pattern match on a `Money` value to inspect its id directly. What happens?
- Construct a `Wallet` value where some id is *removed* from the spent set, "un-spending" a previously spent `Money`. What happens?

If any of these succeed, the abstraction is leaky. Why does the export list prevent them?

Part B — Scala with opaque types

Translate the interface and implement it

Translate the `Wallet` interface your group designed in Part A into Scala using opaque types. Both `Wallet` and `Money` should be opaque to clients.

```
object Wallets:
  opaque type Wallet = ???
  opaque type Money = ???

  // operations: empty, mint, spend, ...
  ...
```

The interface is the same as in Haskell, but you will need to make some idiomatic choices about how to express it in Scala. In a one-line comment next to each choice, write down why you made it. In particular:

- *Functional style or extension methods?* Standalone functions like `mint(w, 100)`, or extension methods like `w.mint(100)`? The two are equivalent in expressive power but read very differently at the call site.

Try to break it

From *outside* the `Wallets` object, attempt:

a. Treat a `Money` as the underlying tuple or case-class and inspect its fields directly.

b. Construct a `Wallet` with arbitrary internal state.

What does the compiler say? Then compare the two enforcement mechanisms:

- In Haskell, the abstraction is enforced by the *export list* of the module: data constructors that are not exported are simply not in scope at the client.
- In Scala, the abstraction is enforced by the `opaque` modifier: outside the defining `object`, the type is treated as abstract and its representation is invisible to the type checker.

Which of the two feels more stringent? Which more ergonomic? Are there things one mechanism allows that the other does not?

Part C — Reflection

Discuss in your group:

a. Why is `Wallet` the more *security-critical* abstraction here? What would happen if `Wallet`'s representation were public but `Money`'s were hidden?

b. The wallet has to be threaded through every operation that changes its state. What is awkward about this? What does it suggest about how purely functional code handles "stateful" abstractions in practice?

c. Suppose you wanted to switch the internal representation of the spent-id collection from a `[Int]` (linear lookup) to a `Set Int` (logarithmic lookup). How much client code would need to change? How does this compare with the same change in a language without encapsulation?

d. Could you achieve the same no-double-spending guarantee using only ADTs (sum types and product types) without encapsulation? What would be different?